

PWM呼吸灯

PWM呼吸灯

一、学习目标

PWM原理

PWM基本参数

二、硬件搭建

三、实验步骤

1. 打开SYSCONFIG配置工具

2. PWM参数配置

3. 写入程序

4. 编译

四、程序分析

五、实验现象

一、学习目标

1.学习PWM呼吸灯原理。

2.通过 TIMG6_C0 输出一个 PWM 信号，控制 LED 指示灯由暗变亮，再由亮变暗，类似于人的呼吸。

PWM原理

PWM 是 Pulse Width Modulation 的缩写，中文意思就是脉冲宽度调制，简称脉宽调制。它是利用微处理器的数字输出来对模拟电路进行控制的一种非常有效的技术，其控制简单、灵活和动态响应好等优点而成为电力电子技术最广泛应用的控制方式，其应用领域包括测量，通信，功率控制与变换，电动机控制、伺服控制、调光、开关电源，甚至某些音频放大器，因此学习 PWM 具有十分重要的现实意义。

其实我们也可以这样理解，PWM 是一种对模拟信号电平进行数字编码的方法。通过高分辨率计数器的使用，方波的占空比被调制用来对一个具体模拟信号的电平进行编码。PWM 信号仍然是数字的，因为在给定的任何时刻，满幅值的直流供电要么完全有(ON)，要么完全无(OFF)。电压或电流源是以一种通(ON)或断(OFF)的重复脉冲序列被加到模拟负载上去的。通的时候即是直流供电被加到负载上的时候，断的时候即是供电被断开的时候。只要带宽足够，任何模拟值都可以使用 PWM 进行编码。

PWM基本参数

PWM是脉冲宽度调制，具有两个非常重要的参数：频率和占空比。

- **频率：**PWM的频率是整个周期的倒数，即单位时间内PWM信号重复的次数。它表示每秒钟完成多少次信号周期。频率决定了PWM信号变化的速率。频率越高，信号周期越短，变化的次数越多。
- **占空比：**占空比是指一个周期内高电平所占的比例。通常以百分比表示。它反映了PWM信号的"宽度"，即高电平部分持续的时间与整个周期时间的比例。占空比决定了信号的平均输出电压。占空比越大，输出的平均电压越高。

二、硬件搭建

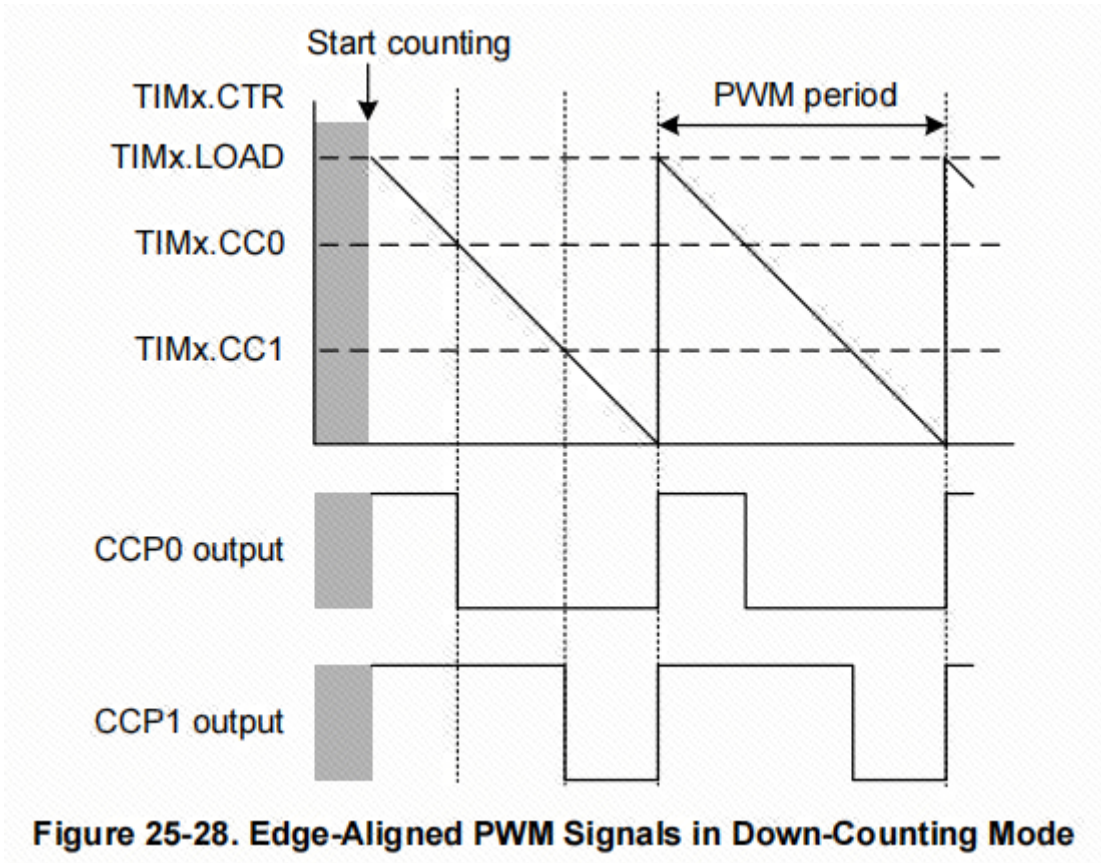
前面我们介绍了MSPM0G系列一共有7个定时器，可以分为2种类型，通用计时器（TIMG）和高级控制计时器（TIMA）。而PWM功能就是在定时器的基础上实现的。从用户手册上可以了解到，MSPM0G3507有7个定时器，哪一个定时器具有PWM功能需要参考数据手册的引脚说明，每一个PWM通道都对应单片机的一个管脚，这个引脚不是唯一固定的，可能有一个或者两个管脚都对应同一个通道。比如说TIMG_C0对应PA5和PA12，就是说PA5和PA12管脚都可以配置为定时器的通道0，我们在使用的时候可以任选其一进行配置。

10	PA5	HFXIN	TIMG8_C0 [2] / SPI0_PICO [3] / TIMA_FAL1 [4] / TIMG0_C0 [5] / TIMG6_C0 [6] / FCC_IN [7]	45	11	9	12	标准
34	PA12		UART3_CTS [2] / SPI0_SCK [3] / TIMG0_C0 [4] / CAN_TX [5] / TIMA0_C3 [6] / FCC_IN [7]	5	27	16	-	高速

本案例使用LED1的 PB2 引脚，配置为定时器6的通道0，产生一个PWM 信号控制灯的亮灭。

15	PB2		UART3_TX [2] / UART2_CTS [3] / I2C1_SCL [4] / TIMA0_C3 [5] / UART1_CTS [6] / TIMG6_C0 [7] / TIMA1_C0 [8]	50	14	-	-	标准
----	-----	--	--	----	----	---	---	----

PWM一共有两种模式，一个是边缘对齐PWM，指的是定时器使用的向下计数模式，输出通道在定时器计数开始的时候会输出高电平，当计数值与输出通道的比较值（图中的CC0和CCP0，CC1和CCP1）相同的时候，输出通道会输出低电平，一直到定时器的计数值计数到0，就完成了周期的脉冲输出。



另一个模式是中心对齐模式，指的是定时器的上下计数模式，输出通道会在定时器开始的时候输出低电平，在定时器向上计数和向下计数两次到达输出通道的比较值相等的时候各翻转一次电平。

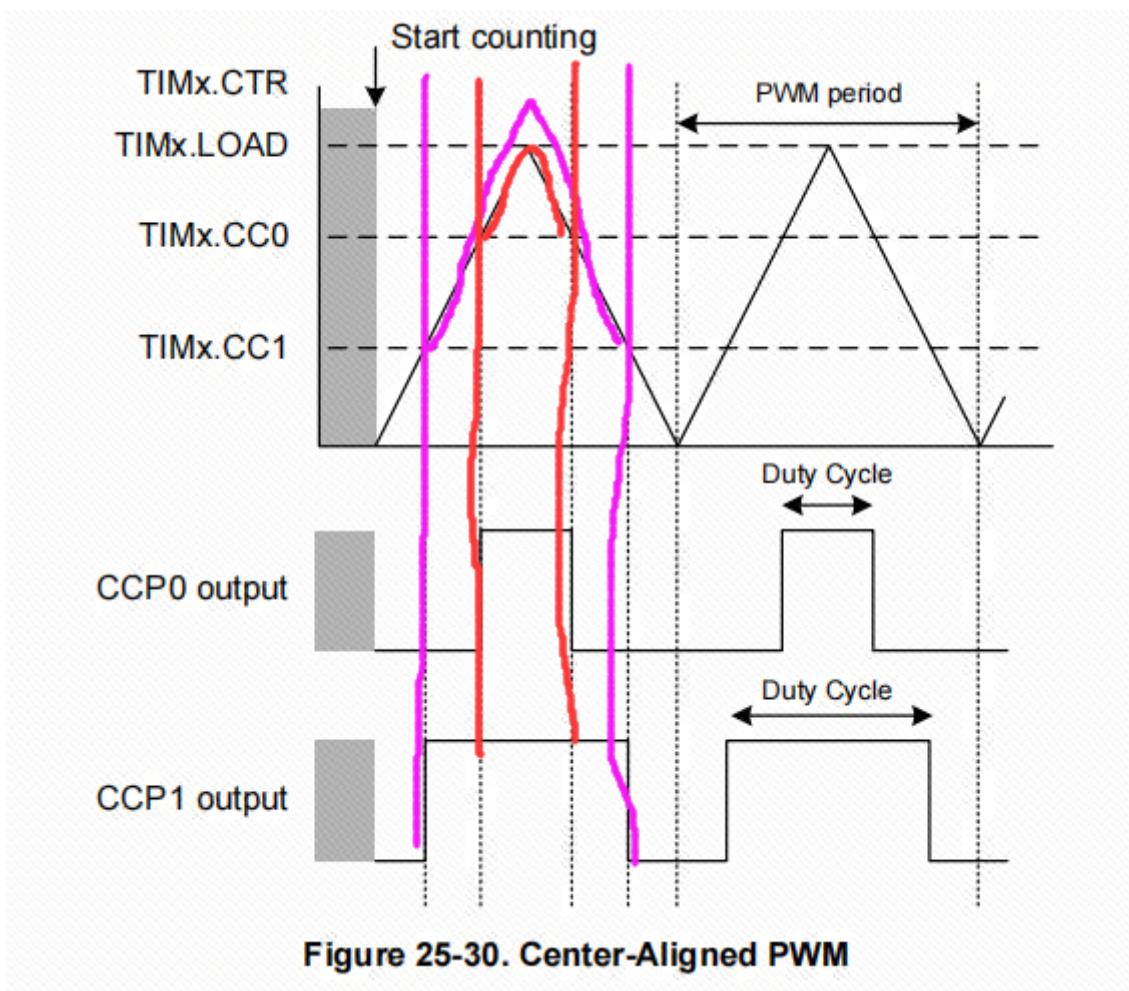


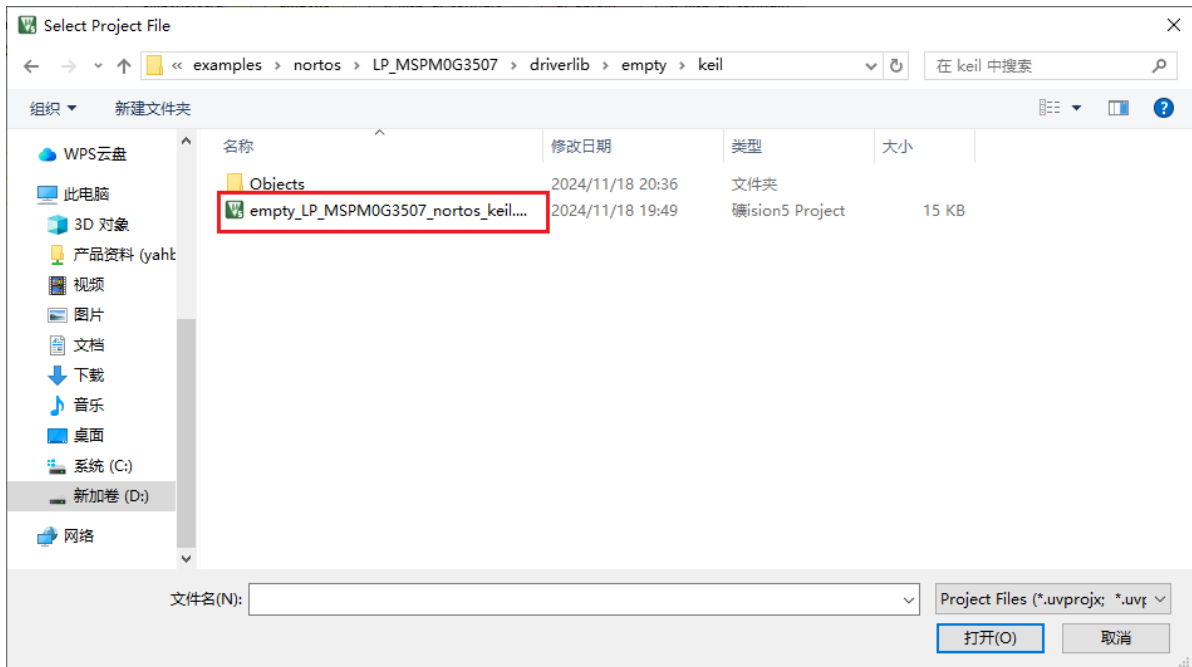
Figure 25-30. Center-Aligned PWM

三、实验步骤

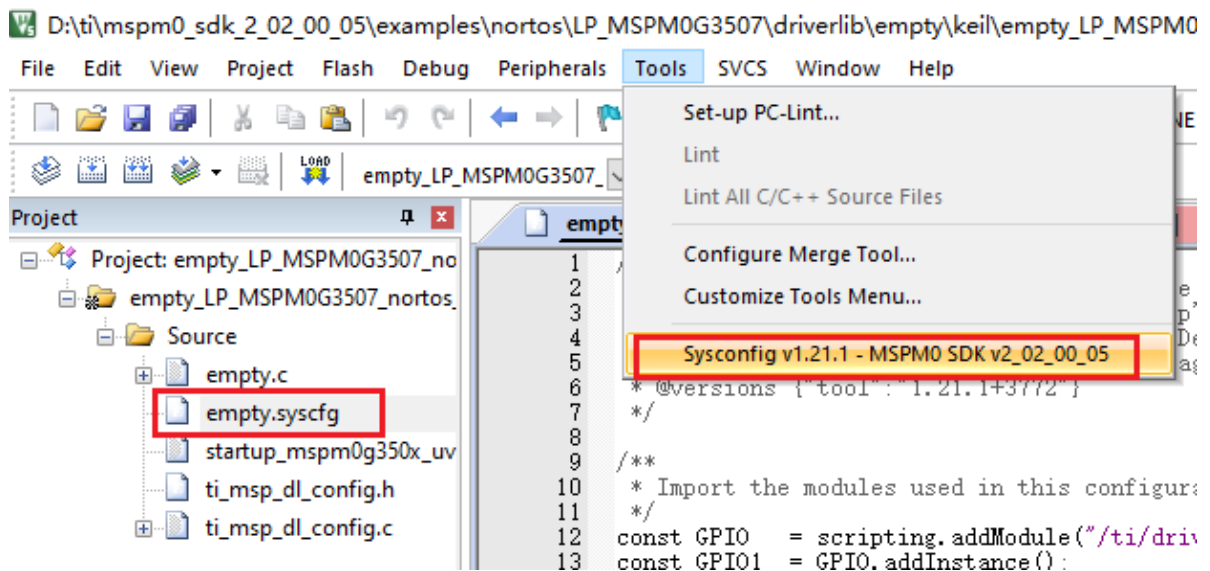
通过驱动PWM信号实现呼吸灯效果，通过改变PWM信号的占空比，可以调节流过LED的电流，从而实现LED亮度的渐变。一般而言，人眼对于80Hz以上的刷新频率没有明显的闪烁感，因为频率较高时无法察觉闪烁。占空比越大，LED越亮；占空比越小，LED越暗。因此，在频率固定的情况下，通过调整占空比可以控制LED的亮度，进而实现呼吸灯效果。本案例将使用板载的LED灯（PB2）实现呼吸灯效果，逐渐亮再逐渐灭。

1. 打开SYSCONFIG配置工具

在KEIL中打开SDK中的空白工程 empty。



选中之后打开，在keil界面打开 empty.syscfg 文件，在 **empty.syscfg** 文件打开情况下，再在 Tools 栏选择打开 SYSCONFIG 的GUI界面。



在 SYSCONFIG 中，左侧选择 MCU 外设，找到 TIMER-PWM 选项并点击进入。在 TIMER-PWM 中点击 ADD，添加定时器下的 PWM 外设。

FILEABOUT

Type Filter Text...

PROJECT CONFIGURATIO...

Project Config... 1/1 ✓ +

MSPM0 DRIVER LIBRARY ...

SYSTEM (9)

Board 1/1 ✓ +

Configuration NVM +

DMA +

GPIO 2 ✓ +

MATHACL +

RTC +

SYSCTL 1/1 ✓ +

SYSTICK 1/1 ✓ +

WWDTC +

ANALOG (6)

ADC12 +

COMP +

DAC12 +

GPAMP +

OPA +

VREF +

COMMUNICATIONS (6)

I2C +

I2C - SMBUS +

MCAN +

SPI +

UART 1/4 ✓ +

UART - LIN +

TIMERS (6)

TIMER 1/7 ✓ +

TIMER - CAPTURE +

TIMER - COMPARE +

1 TIMER - PWM +

TIMER - QEI +

Timer Fault +

Software ▶ TIMER - PWM

TIMER - PWM (0 of 7 Added)

2 + ADD REMOVE ALL

Description

The **Pulse Width Modulation (PWM) Driver module** allows you generate edge- output waveforms.

Under *Basic Configuration* users can:

- Configure clock source
- Configure timer settings
- Configure PWM:
 - Select PWM mode
 - Configure PWM Channels (compare values, duty cycle, etc.)

Under *Advanced Configuration* users can:

- Configure cross trigger

This module supports the different types of Timers included in MSPM0 includi
The following table includes a description of the features included in **MSPM0C**
Check the datasheet of your selected device for more detailed information.

TIM Name	Power Domain	Resolution	Prescaler	Repeat	CC Ch	Phase Load
TIMG0	PD0	16-bit	8-bit	-	2	-
TIMG6	PD1	16-bit	8-bit	-	2	-
TIMG7	PD1	16-bit	8-bit	-	2	-
TIMG8	PD0	16-bit	8-bit	-	2	-

Click the **Add** button to add a TIMER - PWM to your design

Name PWM_0

Selected Peripheral

Quick Profiles ^

PWM Profiles Custom v

2. PWM参数配置

这里设置通道0为4kHz的频率，0%占空比的PWM，周期计数值为1000，比较值也设置为1000，这样的话从1000开始向下计数，每遇到一次1000，PWM的状态就翻转一次。

PWM_LED

Peripheral does not retain register contents in STOP or STANDBY modes. User should take care to save and restore register configuration in application. See Retention Configuration section for more details.

Name	PWM_LED
Selected Peripheral	TIM6

Quick Profiles

PWM Profiles
 Custom

Basic Configuration

Clock Configuration

Timer Clock Source
 BUSCLK

Timer Clock Divider
 Divided by 8

Calculated Timer Clock Source
 4000000

Timer Clock Prescale
 1

Calculated Timer Clock Values

Calculated Clock Frequency (Hz)
 4000000

Timer Clock Information
 61.04 Hz to 2.00 MHz w/ resolution of 250.00 ns

PWM Period Count	1000
Calculated PWM Frequency (Hz)	4000
Start Timer	☒

PWM Configuration

PWM Mode
 Edge-aligned Down Counting

PWM Channel(s)
 PWM Channel 0

Enable Channel Complimentary Output
 None

PWM Channel 0 Channel Specific Configurables for PWM Channel 0

Initial Value
 Low

Counter Compare Value
 1000

Desired Duty Cycle (%)
 0

Actual Duty Cycle (%)
 0

Invert Channel
 ☐

Channel Update Mode
 Capture Compare value has immediate effect

参数说明:

Name: 自定义名称。这里设置为 PWM_LED

PWM Profiles: 配置PWM模版，选择默认的即可。

Timer Clock Source: 定时器的时钟来源，选择 BUSCLK（总线时钟），默认频率为 32MHz。

Timer Clock Divider: 定时器时钟分频器设置分频为 8 分频。

Calculated Timer Clock Source: 显示分频后的时钟频率。

Timer Clock Prescale: 定时器的预分配，我们配置的是TIM6，16位定时器，是支持预分频的。这里不设置预分频。

Calculated Clock Frequency (Hz): 显示分频再预分频后的频率。

PWM Period Count: PWM的周期计数值，这里我们设置为1000。

Calculated PwM Frequency(Hz): 软件自动计算PWM的频率，显示为4000。

Start Timer: 是否开启定时器。这里勾选上，上电开启定时器PWM输出。

PWM Mode: PWM模式。配置为 `Edge-aligned Down Counting` 边缘对齐模式并且是向下计数模式。

PWM Channel(s): PWM的通道。配置为只开启通道0。

Enable Channel Complimentary Output: 是否开启通道互补输出。这里不开启。

Initial Value: PWM通道的默认输出状态。配置为默认的低电平输出。

Counter Compare Value: PWM的比较值。配置为1000，即PWM占空比的设置范围是0-999。

Desired Duty cycle (%): PWM的占空比。这里使用默认的占空比0。

Actual Duty cycle (%): 显示软件自动计算的实际占空比。

Invert Channel: 是否开启翻转输出。这里不开启。

Channel Update Mode: 通道更新模式。配置为 `Capture Compare value has immediate effect` 达到比较值立即更新。

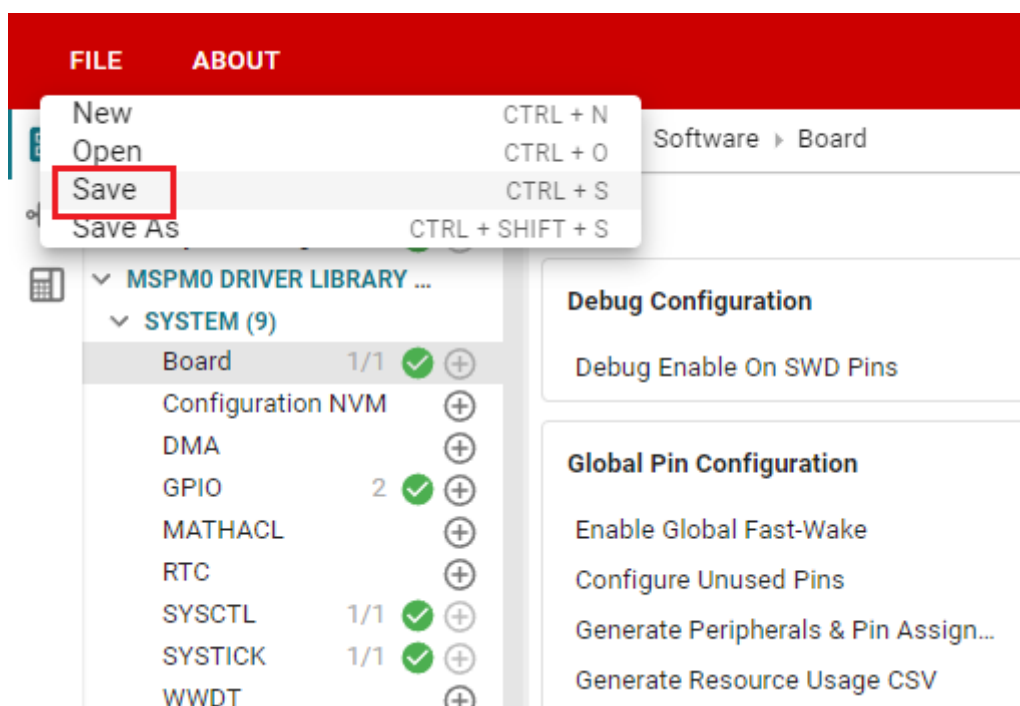
PinMux Peripheral and Pin Configuration		^
Timer Peripheral	TIM6	▼ 🔒
Capture/Compare Pin 0	PB2/50	▼ 🔒

LED1的控制引脚为PB2，PB2有一个定时器6通道0，所以这里选择 `TIM6`，`PB2/50`。

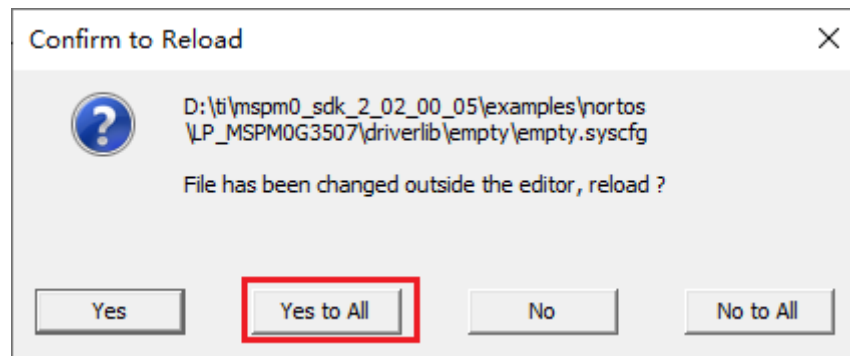
GPIO (1 Added) ⓘ		⊕ ADD	🗑️ REMOVE ALL
✓ KEY			
Name	KEY		
Port	PORTA		
Port Segment	Any		

我们已经将LED1的PB2引脚给PWM占用了，不能再用于GPIO外设上，需要去除。目前例程中GPIO外设仅剩一个按键的配置。

点击 **SAVE** 将 **SYSCONFIG** 中的配置保存，之后关掉 **SYSCONFIG** 回到keil。



在弹窗中选择 `Yes to All`



同样我们还需要确认ti_msp_dl_config.c和ti_msp_dl_config.h文件是否更新。直接编译，编译就会自动更新到keil中。如果没有更新，我们也可以把 SYSCONFIG 中的文件内容复制过来。

3. 写入程序

在empty.c文件中，写入以下代码

```
#include "ti_msp_dl_config.h"

volatile unsigned int delay_times = 0;

//搭配滴答定时器的精准毫秒级延时 Precise millisecond delay with tick timer
void delay_ms(unsigned int ms)
{
    delay_times = ms;
    while( delay_times != 0 );
}

int main(void)
{
    int i = 0;

    SYSCFG_DL_init();

    while (1)
    {

        // 呼吸灯渐亮过程 Breathing light gradually brightens
        for (i = 0; i <= 999; i++)
        {
            // 设置 LED 亮度 Setting LED Brightness

            DL_TimerG_setCaptureCompareValue(PWM_LED_INST,i,GPIO_PWM_LED_C0_IDX);
            delay_ms(1); // 延迟以控制亮度变化速度 Delay to control the speed of
            brightness change
        }
        // 呼吸灯渐暗过程 Breathing light dimming process
        for (i = 999; i > 0; i--)
        {
            // 设置 LED 亮度 Setting LED Brightness

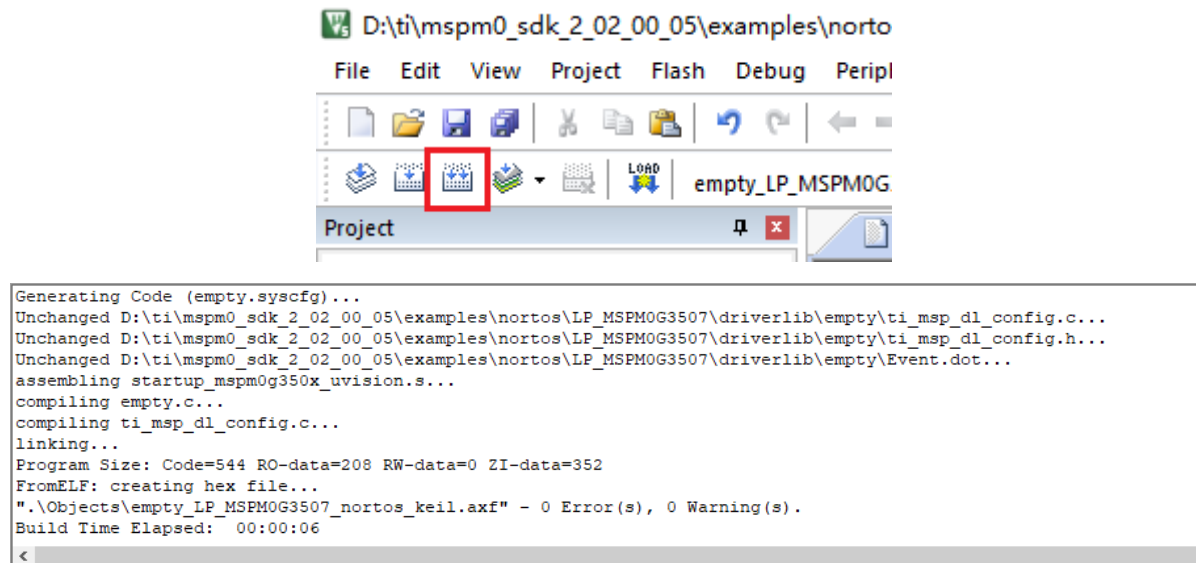
            DL_TimerG_setCaptureCompareValue(PWM_LED_INST,i,GPIO_PWM_LED_C0_IDX);
            delay_ms(1); // 延迟以控制亮度变化速度 Delay to control the speed of
            brightness change
        }
    }
}
```



```
//滴答定时器的中断服务函数 Tick ??timer interrupt service function
void SysTick_Handler(void)
{
    if( delay_times != 0 )
    {
        delay_times--;
    }
}
```

4. 编译

点击 Rebuild 图标。出现以下提示表示编译完成，并且没有错误。



四、程序分析

- empty.c

```
int main(void)
{
    int i = 0;

    SYSCFG_DL_init();

    while (1)
    {
        // 呼吸灯渐亮过程 Breathing light gradually brightens
        for (i = 0; i <= 999; i++)
        {
            // 设置 LED 亮度 Setting LED Brightness

            DL_TimerG_setCaptureCompareValue(PWM_LED_INST,i,GPIO_PWM_LED_C0_IDX);
            delay_ms(1); // 延迟以控制亮度变化速度 Delay to control the speed of
            brightness change
        }
        // 呼吸灯渐暗过程 Breathing light dimming process
        for (i = 999; i > 0; i--)
        {
            // 设置 LED 亮度 Setting LED Brightness
```

```
DL_TimerG_setCaptureCompareValue(PWM_LED_INST,i,GPIO_PWM_LED_C0_IDX);
    delay_ms(1); // 延迟以控制亮度变化速度 Delay to control the speed of
    brightness change
}
}
}
```

初始化硬件之后，进入呼吸灯循环。使用PWM信号的占空比控制LED灯的亮度，呼吸灯 渐亮 和 渐暗 使用两个 for 循环，分别将 PWM 占空比从 0 到 999 再从 999 到 0 逐步调整。delay_ms(1) 设置亮度变化间隔为 1 毫秒，可以通过调整延时函数的参数来控制呼吸灯变化速度。

五、实验现象

程序下载完成之后，可以看到LED灯慢慢亮灭，呈呼吸灯效果。

